

# Hackathon Cohorte 12 Generation

## Proyecto: Tienda en Línea

**TrendyShop** era una tienda de ropa y accesorios muy popular entre los jóvenes de Colombia. Fundada en 2018, llegó a tener 5 tiendas físicas en Bogotá y Medellín, y un sitio web que prometía envíos en 48 horas. Sin embargo, en 2025 la empresa entró en crisis.

## ¿Qué le pasó a TrendyShop?

### Razones del fracaso:

#### Error 1 - Competencia de gigantes

Como le pasó a Dafiti en Chile, TrendyShop no pudo competir con plataformas como Mercado Libre, Falabella y la llegada de marcas asiáticas como Shein y Temu que ofrecían precios mucho más bajos.

#### Error 2 - Mala experiencia de usuario en su tienda online

Al igual que la perfumería Druni, TrendyShop acumuló cientos de quejas por pedidos que tardaban semanas en llegar, productos que no coincidían con lo ofrecido y un servicio al cliente que nunca respondía.

#### Error 3 - Desconfianza en su canal digital

La marca no invirtió en seguridad, en un buen diseño ni en una experiencia de compra fluida. Los clientes desconfiaban de la tienda online, que parecía una "tienda ficticia" con políticas de devolución confusas y pocos datos de contacto reales.

## El Desafío de la Hackathon

**TrendyShop** está en quiebra, pero tiene una última oportunidad: lanzar su ecommerce en 7 horas.

El dueño de la tienda ha contratado a tu equipo para que construyan una tienda en línea moderna que pueda:

**Recuperar la confianza de los clientes:** Mostrar información clara y transparente.

**Ofrecer una experiencia de compra excepcional:** Catálogo atractivo, carrito funcional y persistente.

**Competir con los gigantes:** Ser responsiva, rápida y fácil de usar.

**Contar con un canal de comunicación:** Formulario de contacto funcional y feedback inmediato.

## Descripción del Proyecto

Construir una tienda en línea funcional utilizando HTML, CSS y JavaScript vanilla. La tienda debe ser responsiva, permitir navegar por diferentes secciones y gestionar un carrito de compras persistente con localStorage.

## Objetivos del Proyecto

- Aplicar todos los conceptos de frontend vistos hasta el momento.
  - Construir una interfaz de usuario funcional y responsiva.
  - Implementar un carrito de compras completo.
  - Persistir los datos del carrito entre sesiones del navegador.
  - Modular el código para que sea mantenible y escalable.
- 

## Requisitos Funcionales

### 1. Páginas o Secciones

La tienda debe tener las siguientes secciones, accesibles desde una barra de navegación:

- Página de Inicio: Presentación de la tienda.
- Catálogo: Lista de productos con imagen, nombre, precio y un botón de agregar al carrito.
- Quiénes Somos: Información sobre la tienda.
- Contactanos: Un formulario de contacto con campos de nombre, email y mensaje.

### 2. Catálogo

- Los productos deben mostrarse en formato de tarjetas (cards).
- Cada producto debe tener: imagen, nombre, precio y botón de agregar al carrito.
- Al hacer clic en agregar, el producto debe añadirse al carrito.

### 3. Carrito de Compras

- Un icono o enlace en la barra de navegación que muestre la cantidad de productos.
- Al hacer clic en el icono, debe mostrarse un desplegable o una página con los productos agregados.
- En el carrito se debe ver: nombre, cantidad, precio unitario y subtotal.

- Debe permitir eliminar productos del carrito.
- Debe calcular y mostrar el total de la compra.
- Los datos del carrito deben persistir al recargar la página.

## 4. Formulario de Contacto

- Validar que los campos no estén vacíos.
  - Mostrar un mensaje de confirmación al enviar el formulario.
- 

# Requisitos Técnicos

## 1. Tecnologías Obligatorias

- HTML5
- CSS3
- JavaScript (ES6+)

## 2. Tecnologías Opcionales

- Bootstrap (para estilizado rápido)
- Consumo de API para obtener los productos (Fake Store API, DummyJSON, etc.)

## 3. Responsividad

- El sitio debe verse correctamente en dispositivos móviles, tabletas y escritorio.
- Usar flexbox, grid o el sistema de Bootstrap.

## 4. Estructura de Archivos

### Opción 1

El proyecto debe tener la siguiente estructura:

```
tienda/  
|  
├─ index.html  
├─ quienes-somos.html  
├─ contactanos.html  
|  
├─ css/  
|   └─ style.css  
|  
└─ js/
```

```

|   ├── app.js           # Archivo principal
|   ├── productos.js     # Datos de productos o funciones de API
|   ├── carrito.js       # Lógica del carrito
|   └── ui.js            # Funciones de renderizado
|
└── assets/
    └── images/          # Imágenes de los productos

```

## Opción 2

El proyecto debe tener la siguiente estructura:

```

tienda/
|
├── index.html           # Página de inicio
├── catalogo/
|   └── index.html       # Catálogo de productos
├── quienes-somos/
|   └── index.html       # Información de la tienda
├── contactanos/
|   └── index.html       # Formulario de contacto
├── carrito/
|   └── index.html       # Carrito de compras
|
├── css/
|   └── style.css        # Estilos globales
|
├── js/
|   ├── app.js          # Controlador principal
|   ├── carrito.js      # Modelo del carrito
|   ├── ui.js           # Funciones de renderizado
|   └── productos.js     # Datos de productos
|
├── services/
|   ├── apiService.js   # Cliente HTTP
|   ├── productService # Servicio de productos
|   └── cartService.js   # Servicio del carrito
|
├── assets/
|   └── images/
|       ├── logo.png
|       ├── favicon.ico
|       └── productos/
|           ├── producto1.jpg
|           └── producto2.jpg

```

## Resumen de responsabilidades por capa

| Capa        | Archivos                   | Responsabilidad           |
|-------------|----------------------------|---------------------------|
| Vista       | *.html , style.css , ui.js | Estructura y presentación |
| Modelo      | productos.js , carrito.js  | Datos y lógica de negocio |
| Controlador | app.js                     | Orquestación y eventos    |
| Servicios   | services/*.js              | Comunicación con APIs     |

## 5. Modularidad

- El código debe organizarse en módulos.
- Separar la lógica de negocio de la lógica de presentación.
- Usar funciones reutilizables.

## 6. Persistencia

- Utilizar localStorage para guardar el carrito.
- Cargar el carrito al iniciar la página.
- Actualizar el carrito en localStorage cada vez que se modifique.

---

## Buenas Prácticas a Implementar

### 1. Código Limpio

- Usar nombres descriptivos para variables y funciones.
- Mantener las funciones cortas y con una sola responsabilidad.
- Agregar comentarios solo cuando sea necesario para explicar lógica compleja.

### 2. Organización del Código

- Modularizar el código según la estructura propuesta.
- No escribir todo en un solo archivo.
- Separar la lógica de datos de la lógica de presentación.

### 3. Manejo de Errores

- Validar entradas del usuario (formularios).
- Manejar errores al consumir APIs (try/catch).
- Mostrar mensajes claros al usuario cuando ocurra un error.

## 4. Experiencia de Usuario

- Proporcionar retroalimentación visual al agregar productos al carrito.
- Mostrar mensajes de carga mientras se obtienen datos de la API.
- Asegurarse de que la navegación sea intuitiva.

## 5. Mantenibilidad

- Evitar código duplicado.
- Usar funciones y variables reutilizables.
- Documentar las funciones principales con comentarios.

---

## Criterios de Evaluación

| Criterio                                         | Puntos |
|--------------------------------------------------|--------|
| Cumplimiento de requisitos funcionales           | 30%    |
| Responsividad y diseño                           | 15%    |
| Uso correcto de localStorage                     | 15%    |
| Modularidad y organización del código            | 15%    |
| Calidad del código (limpieza y buenas prácticas) | 15%    |
| Funcionalidad del carrito de compras             | 10%    |

---

## Entregables

- Repositorio de GitHub con el código completo.
- URL del proyecto desplegado (GitHub Pages o similar).
- README.md con instrucciones de instalación y uso.

---

## Recomendaciones Adicionales

- Priorizar la funcionalidad sobre el diseño.
- Completar las secciones principales antes de añadir funcionalidades extra.
- Realizar pruebas constantes.
- Pedir feedback a compañeros durante el desarrollo.

## Estructura del Tablero Scrum

| Columna                       | Descripción                                |
|-------------------------------|--------------------------------------------|
| <b>Backlog</b>                | Tareas planificadas pero no iniciadas      |
| <b>To Do (Sprint Backlog)</b> | Tareas seleccionadas para el Sprint actual |
| <b>In Progress</b>            | Tareas en desarrollo                       |
| <b>Code Review</b>            | Tareas en revisión por otro miembro        |
| <b>Done</b>                   | Tareas completadas y verificadas           |